

SOC HANDS-ON WORKSHOP

SESIÓN 03

Detection Engineering

Afina reglas · Elimina ruido · Construye detecciones que importan

CON INVITADO · sesión especial

03

Perspectiva del invitado

Un profesional en el campo comparte su experiencia real



INVITADO

Nombre · Rol · Empresa

[a completar por el facilitador]

¿Qué veremos hoy?

00:00

Recap S02 + bienvenida al invitado

MITRE, automation rules, contención — dónde quedamos

00:10

¿Qué es Detection Engineering?

La disciplina que transforma alertas en inteligencia operativa

00:30

Ejercicio 06 · Tuning de regla port scan

Reducir 200 alertas/día a las que realmente importan

01:00

Ejercicio 07 · Detección anómala de identidad Okta

Construir una regla sobre comportamiento inusual de MFA

01:30

Ejercicio 08 · Watchlists como contexto de detección

Enriquecer reglas con listas de activos críticos y usuarios VIP

01:50

Cierre · Q&A con invitado · preview S04

Reflexión grupal · preguntas · link registro Sesión 04

Recap sesión anterior

- ✓ MITRE ATT&CK mapeado
- ✓ Cobertura visualizada
- ✓ Automation Rule activa
- ✓ Cross-platform isolation
- ✓ Perspectiva del invitado

¿Qué es Detection Engineering?

La disciplina que convierte Sentinel de una herramienta de alertas en un sistema de inteligencia operativa

El problema

200+

alertas diarias que nadie puede investigar manualmente

60-70%

de alertas son falsos positivos sin tuning

Horas

perdidas en alertas que no significan nada

Ruido

que enmascara el ataque real que necesitas detectar

Detection Engineering



Analizar la regla

Entender por qué dispara tanto — umbral, scope, contexto



Ajustar condiciones

Agregar excepciones, acotar IPs, definir umbrales inteligentes



Enriquecer con contexto

Watchlists de activos críticos, usuarios VIP, IPs confiables



Validar sin romper

Probar en modo simulation antes de activar en producción

La regla que grita todo el tiempo

Port scan detection — la regla más común y más ruidosa en cualquier SOC

Regla original — sin tuning

```
// Umbral demasiado bajo
PaloAltoFirewall_CL
| where EventType_s == 'TRAFFIC'
| summarize count() by SourceIP_s,
    bin(TimeGenerated, 5m)
| where count_ > 10 ← muy bajo
```

- ✗ Dispara 200+ veces al día
- ✗ Incluye escaneos legítimos
- ✗ El analista deja de revisarla
- ✗ El ataque real se esconde

Regla después del tuning

```
PaloAltoFirewall_CL
| where SourceIP_s !in (trusted_ips)
| summarize dcount(DestinationPort_d),
    count() by SourceIP_s,
    bin(TimeGenerated, 10m)
| where dcount_DestinationPort_d > 15
```

- ✓ 3-5 alertas/día de alta fidelidad
- ✓ IPs legítimas excluidas vía watchlist
- ✓ Analista confía en cada alerta
- ✓ El ataque lateral es inmediatamente visible

"Microsoft aprendió que los hunters creaban queries manuales para encontrar señales entre miles de millones de eventos. La clave: capturar incidentes y aprender de ellos hasta encontrar el umbral preciso entre cobertura y fidelidad (signal-to-noise)."

EJ. 06

Tuning de regla port scan en Palo Alto

Reducir falsos positivos · Mejorar fidelidad · Mantener cobertura real

Objetivo

Modificar la regla de detección de port scan para reducir los falsos positivos, excluyendo rangos de IP legítimos via watchlist y ajustando el umbral de puertos únicos escaneados.

Dónde lo harás

Defender portal
→ Microsoft Sentinel
→ Analytics
→ [regla port scan]

Los 3 cambios que harás en la regla

01

Agregar watchlist de IPs de confianza

| where SourceIP_s !in (_GetWatchlist('TrustedNetworks') | project SearchKey)

02

Cambiar umbral a puertos únicos

| summarize dcount(DestinationPort_d) | where dcount_DestinationPort_d > 15

03

Extender ventana de análisis (5→10 min)

| where TimeGenerated > ago(10m) — atacante lento activa la alerta

Watchlists: el contexto que le falta a tus reglas

Datos de referencia en Sentinel · disponibles en cualquier query KQL · actualizables sin tocar la regla

📁 Activos críticos

Lista de servidores de producción, sistemas financieros, controladores de dominio — para priorizar alertas.

```
_GetWatchlist('CriticalAssets')
```

👤 Usuarios VIP

Ejecutivos, cuentas de servicio privilegiadas — para escalar automáticamente si algo los involucra.

```
_GetWatchlist('VIPUsers')
```

🌐 IPs de confianza

Rangos de infraestructura interna, scanners autorizados — para excluirlos de detecciones ruidosas.

```
_GetWatchlist('TrustedNetworks')
```

🚫 IOCs activos

Hashes maliciosos, dominios de phishing, IPs de C2 — para detectar automáticamente cuando aparecen.

```
_GetWatchlist('MaliciousIOCs')
```

EJ. 07

Detección anómala de identidad en Okta

Detectar MFA manipulation · comportamiento inusual · cuenta comprometida con credenciales válidas

Objetivo

Construir una nueva regla de detección sobre los logs de Okta del Training Lab que identifique múltiples fallos de MFA — señal de MFA fatigue attack o credential stuffing.

Dónde lo harás

Defender portal
→ Sentinel → Analytics
→ Create → Scheduled
Query Rule

Query base — lo que construirás

```
OktaSSO_CL
| where EventType_s has 'MFA'
| where Outcome_Result_s == 'FAILURE'
| summarize
    FailedMFA = countif(EventType_s has 'FAILURE'),
    dcount(DebugData_RequestId_s)
    by UserId_s, bin(TimeGenerated, 30m)
| where FailedMFA > 3
```

Enriquecimiento con watchlists

Agregar contexto de negocio — quién es el usuario y qué tan crítico es el sistema afectado

Objetivo

Crear una watchlist con usuarios de alto privilegio y modificar una regla para que las alertas sobre esos usuarios se generen con severidad Alta — sin importar el umbral original.

Dónde lo harás

Defender portal
→ Sentinel
→ Configuration
→ Watchlists → Add new

KQL con enriquecimiento de watchlist — lo que construirás

```
let VIPUsers = (_GetWatchlist('VIPUsers') | project UserPrincipalName, Department, CriticalityLevel);
OktaSSO_CL
| where EventType_s has 'MFA' and Outcome_Result_s == 'FAILURE'
| lookup kind=leftouter VIPUsers on $left.UserId_s == $right.UserPrincipalName
| extend IsVIP = isnotempty(CriticalityLevel)
| extend AlertSeverity = iff(IsVIP == true, 'High', 'Medium')
| where FailedMFA > iff(IsVIP, 1, 3) // VIP threshold = 1 fallo
| project TimeGenerated, UserId_s, FailedMFA, Department, AlertSeverity
```

"La lógica de enriquecimiento con watchlist es equivalente a una Sigma rule con enrichment context: la detección base detecta el patrón, la watchlist agrega el contexto de negocio que determina la severidad y la respuesta."

¿Qué lograste hoy?

SESIÓN 03 completada



Regla afinada

Port scan pasó de 200+ alertas/día a 3-5 de alta fidelidad



Detección nueva

MFA abuse en Okta — una regla que tú construiste desde cero



Watchlist activa

Usuarios VIP con umbral diferenciado — contexto de negocio en KQL



Detection mindset

Sabes cuándo una alerta es un problema del atacante o de la regla

Próxima sesión → S04: Operations + Data Lake

Optimizarás costos de ingesta con table tiers, configurarás KQL search jobs y detectarás sobre datos históricos en el Data Lake

Registro → meetup.com/msperu/events/314627551